

Aula 05 — Spring Framework

Relacionamentos ð 1:N ð N:N ð Cliente e venda

Mauricio Duailibi Neto

Turma Java Entra21

Como será a aula de hoje

Parte 1 — 18h30 às 20h

- Revisão da Aula 04
- Relacionamento: um cadastro aponta para outro
- 1:N com categoria e material
- N:N: a ideia da lista do meio
- Projeto **laboratorio** — fazemos juntos

Parte 2 — 20h20 às 22h

- Voltamos ao **StockSales**
- Segundo cadastro: cliente
- Primeira venda: um cliente e um produto

Dois projetos nesta aula

Não vamos misturar os códigos.

laboratorio Primeira parte da aula. Aqui aprendemos a ligar categoria e material.

StockSales O sistema da loja, que já existe. Só depois do intervalo.

Quando aparecer um código, o slide vai dizer:

qual projeto e **qual arquivo** estamos mexendo.

Mapa da aula

- 1 Revisão da Aula 04
- 2 Relacionamentos
- 3 Projeto laboratorio
- 4 Categoria (1:N)
- 5 Tela com select
- 6 Desafios
- 7 Parte 2 — StockSales
- 8 Recapitulando

O que já sabemos

Nos dois projetos, até agora, existe **um** model e **um** Controller.

- laboratório: Material — GET, POST, PUT, DELETE
- StockSales: Produto — GET, POST, PUT, DELETE
- A lista fica **na classe** do Controller
- PUT altera pelo id. Não gera id novo. Não dá add
- O JavaScript chama a API com Axios

Cada cadastro vive sozinho.

O material não aponta para ninguém. O produto tampouco.

Os quatro verbos

Ação	Verbo HTTP	O que faz
Listar / ler	GET	Lê, não muda nada
Cadastrar	POST	Cria um item novo
Editar	PUT	Troca os dados de um item
Excluir	DELETE	Apaga um item

Hoje esses verbos continuam iguais.

O que muda é o **conteúdo** do cadastro:
um objeto passa a guardar o id de outro.

O que ainda falta

No laboratorio, o material tem id, nome e quantidade.
Ele não diz de que grupo é.

No StockSales, só existe produto.

A loja ainda não tem cliente e ainda não registra venda.

Pergunta de hoje

Como um cadastro aponta para outro
sem copiar todos os dados?

Cadastros que se falam

Até agora cada lista vivia sozinha:
materiais de um lado, produtos de outro.

No mundo real eles se encaixam:

- O material **pertence** a uma categoria
- Depois, na loja, a venda **pertence** a um cliente
- E aponta para um produto

Essa ligação chama **relacionamento**.

Uma analogia que todo mundo já viu

Uma turma tem vários alunos.

Um aluno está em **uma** turma.

Não copiamos a lista de alunos dentro da turma
toda vez que alguém pergunta o nome da turma.

Guardamos no aluno o número da turma:

```
turmaId = 2.
```

Quando precisamos do nome, procuramos a turma 2
na lista de turmas.

Dois jeitos de ligar

Nome	Significado
1:N	Um de um lado, vários do outro
N:N	Vários dos dois lados

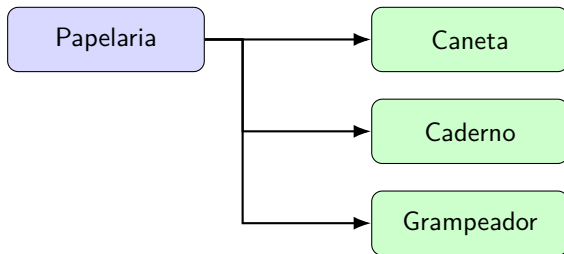
1:N — uma categoria, vários materiais.

Cada material tem **uma** categoria.

N:N — um material tem várias etiquetas,
e a mesma etiqueta serve para vários materiais.

1:N — o desenho

Uma categoria aponta para vários materiais.
O material aponta para **uma** categoria.



Papeleria é o “1”.
Caneta, Caderno e Grampeador são o “N”.

Como gravamos o 1:N

Não colocamos uma lista de materiais dentro da categoria.

Colocamos no material o id da categoria:

Material	quantidade	categoriald
Caneta	10	1
Caderno	5	1
Mouse	3	2

A categoria 1 é Papelaria.

A categoria 2 é Informática.

É o mesmo jeito do turmaId no aluno.

O JSON do material com categoria

O cadastro passa a levar mais um número:

```
{"nome": "Caneta", "quantidade": 10, "categoriaId": 1}
```

O servidor não precisa do texto “Papelaria” nesse JSON.

O id 1 já aponta para ela.

Se a pessoa mudar o nome da categoria depois,
a Caneta continua ligada ao id 1.

O nome novo aparece sozinho.

N:N — por que um id só não basta

Um material pode ser Frágil e Uso interno.
A etiqueta Uso interno serve para Caneta e Caderno.

Se o material tiver um único `etiquetaId`,
ele só cabe em uma etiqueta.

Se a etiqueta tiver um único `materialId`,
ela só cabe em um material.

Os dois lados são “vários”.
Um campo só não resolve.

N:N — a lista do meio

Criamos uma terceira lista.

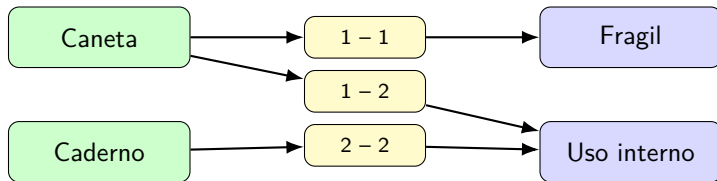
Cada linha é um par: este material + esta etiqueta.

materialId	etiquetaId
1 (Caneta)	1 (Fragil)
1 (Caneta)	2 (Uso interno)
2 (Caderno)	2 (Uso interno)

Caneta aparece duas vezes: tem duas etiquetas.

Uso interno aparece duas vezes: serve para dois materiais.

N:N — o desenho



O amarelo é a lista do meio.
Sem ela, os dois lados não se encontram.

Depois do intervalo, a mesma ideia na loja

Até hoje o StockSales só tem Produto.

Na segunda parte a loja ganha **cliente** e **venda**.

A venda guarda `clienteId` e `produtoId`:
o mesmo 1:N da categoria com o material.

N:N (vários produtos na mesma venda) fica como extra,
se der tempo.

O que não vamos usar hoje

- Banco de dados
- Anotações do tipo `@OneToMany`
- JPA

Continuamos com lista na memória e um `int` guardando o id do outro cadastro.

O raciocínio é o mesmo que o banco vai usar depois.
Só a ferramenta muda.

O projeto laboratorio

Continuamos no projeto **laboratorio**.
Não geramos outro no `start.spring.io`.

O que já deve estar pronto

Material com GET, POST, PUT e DELETE.
Formulário na tela. Botões Editar e Excluir.

Hoje entra um cadastro novo: **categoria**.
O material ganha um `categoriaId`.

Passo 1 — Subir o Spring

Onde estamos

Projeto **laboratorio**

No terminal, dentro da pasta laboratorio:

```
./mvnw spring-boot:run
```

No Windows: `.\mvnw.cmd spring-boot:run`

Abram no navegador: `http://localhost:8080`

Um Spring por vez

Se o StockSales ainda estiver na porta 8080, parem o outro antes.

Quais arquivos já existem

Projeto: **laboratorio**

Arquivo	Para quê
Material.java	Os dados de um material
MaterialController.java	GET, POST, PUT, DELETE
static/index.html	A página
static/js/app.js	O JavaScript da página

Vamos **acrescentar** categoria e a classe Dados.

Onde cada arquivo fica

```
laboratorio/  
  src/main/java/com/entra21/laboratorio/  
    LaboratorioApplication.java  
    Dados.java                (novo)  
    model/Material.java  
    model/Categoria.java      (novo)  
    controller/MaterialController.java  
    controller/CategoriaController.java (novo)  
  src/main/resources/static/  
    index.html  
    js/app.js
```

Por que a classe Dados

Vamos ter dois Controllers: material e categoria.

Se cada um guardar a própria lista, um **não vê** o cadastro do outro.

Por isso uma classe só, Dados, com as listas `static`.

`static` = uma lista só, visível para qualquer classe do projeto.

Até agora a lista ficava dentro do Controller.

Hoje ela sai de lá, porque passou a ter dois Controllers.

Classe Dados

Onde estamos

Projeto **laboratorio**

Arquivo novo:

src/main/java/com/entra21/laboratorio/Dados.java

```
package com.entra21.laboratorio;

import java.util.ArrayList;
import com.entra21.laboratorio.model.Material;
import com.entra21.laboratorio.model.Categoria;

public class Dados {
    public static ArrayList<Material> materiais = new ArrayList<>();
    public static int proximoIdMaterial = 1;
    public static ArrayList<Categoria> categorias = new ArrayList<>();
    public static int proximoIdCategoria = 1;
}
```


Mover a lista de materiais

Onde estamos

`MaterialController.java`

Tirem os campos `materiais` e `proximoId` de dentro da classe.

Em todos os métodos, troquem:

- `materiais` → `Dados.materiais`
- `proximoId` → `Dados.proximoIdMaterial`

O import: `com.entra21.laboratorio.Dados`

Confirmam GET, POST, PUT e DELETE no navegador **antes** de seguir. A lista precisa continuar funcionando.

Classe Categoria

Onde estamos

Arquivo novo:

src/main/java/com/entra21/laboratorio/model/Categoria.java

```
public class Categoria {  
    private int id;  
    private String nome;  
  
    public Categoria() {  
    }  
  
    public Categoria(int id, String nome) {  
        this.id = id;  
        this.nome = nome;  
    }  
}
```

Completem os gets e sets de id e nome.

categoriald no Material

Onde estamos

model/Material.java

Acrescentem o campo e os get/set.

```
private int categoriaId;

public int getCategoriaId() {
    return categoriaId;
}

public void setCategoriaId(int categoriaId) {
    this.categoriaId = categoriaId;
}
```

O material agora aponta para uma categoria.

Construtor cheio do Material

Onde estamos

Ainda em `Material.java`

Se ainda não tiverem, acrescentem este construtor.

Mantenham o vazio. O vazio continua obrigatório no POST.

```
public Material(int id, String nome,  
                int quantidade, int categoriaId) {  
    this.id = id;  
    this.nome = nome;  
    this.quantidade = quantidade;  
    this.categoriaId = categoriaId;  
}
```

CategoriaController — seed e GET

Onde estamos

controller/CategoriaController.java, arquivo novo

```
@RestController
public class CategoriaController {

    public CategoriaController() {
        if (Dados.categorias.isEmpty()) {
            Dados.categorias.add(new Categoria(1, "Papeleria"));
            Dados.categorias.add(new Categoria(2, "Informatica"));
            Dados.categorias.add(new Categoria(3, "Limpeza"));
            Dados.proximoIdCategoria = 4;
        }
    }

    @GetMapping("/api/categorias")
    public ArrayList<Categoria> listar() {
        return Dados.categorias;
    }
}
```

POST de categoria

Onde estamos

Ainda em `CategoriaController.java`

```
@PostMapping("/api/categorias")
public Categoria cadastrar(@RequestBody Categoria categoria) {
    categoria.setId(Dados.proximoIdCategoria);
    Dados.proximoIdCategoria = Dados.proximoIdCategoria + 1;
    Dados.categorias.add(categoria);
    return categoria;
}
```

Testem no navegador:

`http://localhost:8080/api/categorias`

Devem aparecer Papelaria, Informatica e Limpeza.

PUT de material também grava a categoria

Onde estamos

MaterialController.java, método atualizar

Junto de setName e setQuantidade:

```
Dados.materiais.get(i).setName(dados.getNome());  
Dados.materiais.get(i).setQuantidade(dados.getQuantidade());  
Dados.materiais.get(i).setCategoriaId(dados.getCategoriaId());
```

Sem essa linha, o Editar apaga a categoria e deixa categoriaId em 0.

O POST de material já lê o campo novo

Não precisamos de método extra.

O JSON agora traz categoriaId.

O Spring preenche o campo no objeto.

O add na lista já guarda tudo.

Só o JavaScript precisa passar a **enviar** esse número.

Conferindo o backend

`/api/categorias` lista as três categorias

`/api/materiais` continua listando

Um POST de material com `categoriaId` no JSON devolve o campo preenchido

Se `/api/categorias` der 404, o Controller novo não compilou ou o Spring não reiniciou.

A pessoa não precisa decorar o id

Digitar `categoriaId = 2` na mão é fácil de errar.

No HTML existe o `<select>`: uma lista para escolher.

A pessoa vê o **nome** (Papelaria).

O JavaScript manda o **id** (1) no JSON.

O select no HTML

Onde estamos

Projeto **laboratorio**

static/index.html, dentro do form

```
<label>Categoria</label>  
<select id="categoriaId"></select>
```

Fica junto dos inputs de nome e quantidade.
O JS vai encher as opções depois do GET.

Uma lista de categorias no JS

Onde estamos

static/js/app.js, no topo do arquivo

Vamos guardar a resposta do GET.

A lista da tela usa essa variável para achar o nome.

```
var categorias = [];
```

Preencher o select

Onde estamos

static/js/app.js

```
function carregarCategorias() {  
  axios.get("/api/categorias").then(function (resposta) {  
    categorias = resposta.data;  
    var select = document.getElementById("categoriaId");  
    select.innerHTML = "";  
    for (var i = 0; i < categorias.length; i++) {  
      var opcao = document.createElement("option");  
      opcao.value = categorias[i].id;  
      opcao.textContent = categorias[i].nome;  
      select.appendChild(opcao);  
    }  
  });  
}
```

value é o id. O texto visível é o nome.

Por que `value` é o `id`

O que a pessoa vê: Papelaria.

O que o servidor precisa: 1.

O `<option>` guarda os dois:
`value="1"` e o texto Papelaria.

`select.value` devolve "1" — ainda é texto.

Por isso usamos `Number(...)` na hora de montar o JSON.

Achar o nome pelo id

Onde estamos

static/js/app.js

```
function nomeDaCategoria(categoriaId) {  
  for (var i = 0; i < categorias.length; i++) {  
    if (categorias[i].id === categoriaId) {  
      return categorias[i].nome;  
    }  
  }  
  return "";  
}
```

Percorre a lista. Quando o id bate, devolve o nome.
Se não achar, devolve texto vazio.

Mostrar o nome na lista

Onde estamos

static/js/app.js, no for de carregarMateriais

Troquem o `textContent` do `li` por:

```
li.textContent = material.nome  
  + " - quantidade: " + material.quantidade  
  + " - " + nomeDaCategoria(material.categoriaId)  
  + " ";
```

A pessoa lê “Caneta - quantidade: 10 - Papelaria”.

Não lê o número 1.

O submit manda o categoriaId

Onde estamos

static/js/app.js, no objeto dados
do submit — no POST e no PUT

```
var dados = {  
  nome: document.getElementById("nome").value,  
  quantidade: Number(  
    document.getElementById("quantidade").value),  
  categoriaId: Number(  
    document.getElementById("categoriaId").value)  
};
```

Sem Number, o Java pode recusar o campo,
porque espera int.

Editar também preenche o select

Onde estamos

static/js/app.js, no clique de Editar

Além de nome e quantidade:

```
document.getElementById("categoriaId").value =  
    evento.target.getAttribute("data-categoria-id");
```

No botão Editar, acrescentem:

```
btnEditar.setAttribute("data-categoria-id",  
    material.categoriaId);
```

Quando carregar a página

Onde estamos

No final de `app.js`

As categorias precisam chegar **antes** da lista, senão `nomeDaCategoria` não acha o nome.

```
carregarCategorias();  
carregarMateriais();
```

Se o nome da categoria sair vazio na primeira carga, chamem `carregarMateriais` **dentro** do `then` de `carregarCategorias`.

O caminho do cadastro com categoria

- 1 A página pede GET /api/categorias
- 2 O select mostra Papelaria, Informatica, Limpeza
- 3 A pessoa escolhe Papelaria, digita Caneta, envia
- 4 O JS manda categoriaId: 1 no POST
- 5 O material entra na lista com esse número
- 6 Na tela, o JS troca 1 por Papelaria

No Network: um GET de categorias e um POST de material.

Conferindo o 1:N

O select mostra as três categorias

Cadastrar Caneta em Papelaria mostra o nome na lista, não só o id

Editar troca a categoria e não duplica o material

Mouse em Informatica não aparece como Papelaria

No Network, o JSON do POST tem categoriaId

Erros que mais aparecem

- Select vazio — `carregarCategorias` não rodou
- Nome da categoria em branco — a lista de materiais rodou antes do GET de categorias
- `categoriaId` some no Editar — faltou o `setCategoriaId` no PUT
- Lista de materiais some — ainda está no Controller, não em Dados
- Porta 8080 ocupada

Agora é com vocês

Continuem no projeto **laboratorio**.
Mesmos arquivos.

- 1 Filtrar a lista por categoria
- 2 Não apagar categoria que ainda tem material
- 3 Etiquetas: o N:N

Salve → reinicie se precisar → teste no navegador.
F12, aba Network.

Desafio 1 — filtro por categoria

Onde estamos

`index.html` e `static/js/app.js`

Coloquem um segundo select, por exemplo
`id="filtro-categoria"`.

A primeira opção: `value=` e texto “Todas”.

As outras: as mesmas categorias.

Em `carregarMateriais`, se o filtro tiver um id,
só mostrem o material cujo `categoriaId` é aquele.

No `change` do select, chamem `carregarMateriais` de novo.

Desafio 1 — o if do filtro

Ideia do for (você escrevem o resto):

```
var filtro = document.getElementById(  
    "filtro-categoria").value;  
  
if (filtro !== "" &&  
    material.categoriaId !== Number(filtro)) {  
    continue;  
}
```

continue pula para o próximo material
e não cria o li.

Desafio 2 — não apagar categoria em uso

Onde estamos

`CategoriaController.java` e `app.js`

Antes de remover a categoria, percorram `Dados.materiais`.

Se algum material tiver aquele `categoriaId`, lancem `ResponseStatusException` com `HttpStatus.BAD_REQUEST`.

No JS, o clique de excluir categoria usa `catch` e escreve em `#mensagem`:
“Categoria em uso”.

Testem apagar Papelaria com a Caneta ainda cadastrada:
tem que recusar.

Desafio 3 — etiquetas (N:N)

Onde estamos

Arquivos novos no laboratorio

- 1 Classe Etiqueta: id e nome
- 2 Classe MaterialEtiqueta: materialId e etiquetaId
- 3 Listas dessas duas classes em Dados
- 4 GET e POST de /api/etiquetas
- 5 POST /api/materiais/{id}/etiquetas/{etiquetaId} para vincular

Seed sugerido: Fragil e Uso interno.

Desafio 3 — o POST de vincular

O método não cria material nem etiqueta.
Só acrescenta uma linha na lista do meio.

```
@PostMapping("/api/materiais/{id}/etiquetas/{etiquetaId}")
public MaterialEtiqueta vincular(@PathVariable int id,
    @PathVariable int etiquetaId) {
    MaterialEtiqueta ligacao = new MaterialEtiqueta();
    ligacao.setMaterialId(id);
    ligacao.setEtiquetaId(etiquetaId);
    Dados.ligacoes.add(ligacao);
    return ligacao;
}
```

Construtor vazio + gets e sets na classe MaterialEtiqueta.

Desafio 3 — mostrar na lista

No JS, peçam GET das ligações (um GET `/api/ligacoes` ajuda).

Para cada material, percorram as ligações.

Quando o `materialId` bater, achem o nome da etiqueta e cole no texto do `li`.

Um material pode mostrar duas etiquetas.

A mesma etiqueta pode aparecer em dois materiais.

Extra: botão para desvincular, apagando aquela linha da lista do meio — sem apagar o material.

Se terminarem os três, ainda no **laboratorio**:

- Não vincular a mesma etiqueta duas vezes no mesmo material
- Recusar vincular se o material ou a etiqueta não existir
- Página simples só para cadastrar categoria

Checklist

Estou no projeto laboratorio?
As listas estão em Dados?
/api/categorias devolve as três?
O material tem categoriaId?
A lista mostra o **nome** da categoria?
O PUT não zera a categoria?

Retomamos às **20h20**

Fechem o laboratório.

Parem o Spring.

Na segunda parte abrimos o **StockSales**.

Mudamos de projeto

Onde estamos

Agora é o projeto **StockSales**.
O laboratório já está encerrado.

Parem o Spring do laboratório.
Abram a pasta do StockSales e subam de novo.

Onde o StockSales está

Até agora a loja tem **só** produto.

- Um model: Produto
- Um Controller: GET, POST, PUT, DELETE
- A lista fica na classe do Controller
- Tela de produtos com formulário

Ainda **não** tem cliente.

Ainda **não** tem venda.

Ainda **não** tem a classe Dados.

O que vamos fazer no StockSales

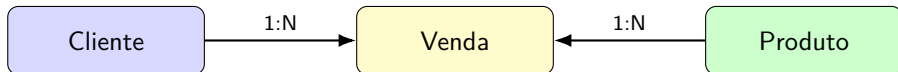
Aplicar o 1:N da primeira parte na loja.

- 1 Classe Dados — as listas num lugar só
- 2 Cadastro de **cliente**
- 3 Cadastro de **venda**: um cliente e um produto

A venda guarda `clienteId` e `produtoId`.

É o mesmo `categoriaId` do material, com outro nome.

O desenho da venda de hoje



Um cliente tem várias vendas.

Um produto aparece em várias vendas.

Cada venda, **nesta aula**, aponta para **um** produto.

Classe Dados no StockSales

Onde estamos

Projeto **StockSales**

Arquivo novo:

src/main/java/com/entra21/stocksales/Dados.java

```
package com.entra21.stocksales;

import java.util.ArrayList;
import com.entra21.stocksales.model.Produto;

public class Dados {
    public static ArrayList<Produto> produtos = new ArrayList<>();
    public static int proximoIdProduto = 1;
}
```

static = uma lista só.

Cliente e venda entram nessa classe daqui a pouco.

Construtor cheio do Produto

Onde estamos

model/Produto.java

Se ainda não tiver, acrescentem. Mantenham o vazio.

```
public Produto(int id, String nome, double preco, int estoque) {  
    this.id = id;  
    this.nome = nome;  
    this.preco = preco;  
    this.estoque = estoque;  
}
```

Sem o vazio, o POST quebra.

Sem o cheio, o seed do construtor fica longo.

Mover a lista de produtos

Onde estamos

ProdutoController.java

Tirem produtos e proximoID da classe.

```
public ProdutoController() {  
    if (Dados.produtos.isEmpty()) {  
        Dados.produtos.add(new Produto(1, "Teclado Mecanico", 250, 12));  
        Dados.produtos.add(new Produto(2, "Mouse Gamer", 120, 30));  
        Dados.produtos.add(new Produto(3, "Monitor 24", 900, 8));  
        Dados.produtos.add(new Produto(4, "Headset", 200, 15));  
        Dados.proximoIdProduto = 5;  
    }  
}
```

Em GET, POST, PUT e DELETE: Dados.produtos
e Dados.proximoIdProduto.

Conferindo depois de mover

A página de produtos abre

Teclado, Mouse, Monitor e Headset aparecem

Cadastrar um produto novo atualiza a lista

O `if (Dados.produtos.isEmpty())` evita duplicar o seed se o Spring criar o Controller de novo.

Classe Cliente

Onde estamos

Arquivo novo:

src/main/java/com/entra21/stocksales/model/Cliente.java

```
public class Cliente {  
    private int id;  
    private String nome;  
    private String email;  
    private String telefone;  
  
    public Cliente() {}  
  
    public Cliente(int id, String nome, String email, String telefone) {  
        this.id = id;  
        this.nome = nome;  
        this.email = email;  
        this.telefone = telefone;  
    }  
}
```

Completem os gets e sets dos quatro atributos.

Clientes em Dados

Onde estamos

Dados.java, acrescentar

```
public static ArrayList<Cliente> clientes = new ArrayList<>();  
public static int proximoIdCliente = 1;
```

Import de Cliente no topo do arquivo.

ClienteController — seed e GET

Onde estamos

Arquivo novo:

controller/ClienteController.java

```
@RestController
public class ClienteController {

    public ClienteController() {
        if (Dados.clientes.isEmpty()) {
            Dados.clientes.add(new Cliente(1, "Ana", "ana@email.com", "1199"));
            Dados.clientes.add(new Cliente(2, "Bruno", "bruno@email.com", "1188"));
            Dados.clientes.add(new Cliente(3, "Carla", "carla@email.com", "1177"));
            Dados.proximoIdCliente = 4;
        }
    }

    @GetMapping("/api/clientes")
    public ArrayList<Cliente> listar() {
        return Dados.clientes;
    }
}
```

POST de cliente

Onde estamos

Ainda em `ClienteController.java`

```
@PostMapping("/api/clientes")
public Cliente cadastrar(@RequestBody Cliente cliente) {
    cliente.setId(Dados.proximoIdCliente);
    Dados.proximoIdCliente = Dados.proximoIdCliente + 1;
    Dados.clientes.add(cliente);
    return cliente;
}
```

Testem `/api/clientes` no navegador:
Ana, Bruno e Carla.

Onde estamos

Arquivos novos:

`static/clientes.html`

`static/js/clientes.js`

Copiem o desenho de produtos:

- Formulário: nome, e-mail, telefone
- Lista na `<ul>`
- `carregarClientes` com GET
- Submit com POST e `preventDefault`

Na home, um link para `/clientes.html`.

O que é uma venda nesta aula

Uma venda tem **um** cliente e **um** produto.

- Quem compra: o cliente — `clienteId`
- O quê: um produto — `produtoId`
- Quantas unidades
- $\text{Total} = \text{preço} \times \text{quantidade}$

Ao confirmar: o estoque daquele produto diminui.
Se não houver estoque, a venda **não** é gravada.

Classe Venda

Onde estamos

Arquivo novo:

src/main/java/com/entra21/stocksales/model/Venda.java

```
public class Venda {  
    private int id;  
    private int clienteId;  
    private int produtoId;  
    private int quantidade;  
    private double total;  
  
    public Venda() {  
    }  
}
```

Construtor vazio + gets e sets dos cinco atributos.

Onde estamos

Dados.java

```
public static ArrayList<Venda> vendas = new ArrayList<>();  
public static int proximoIdVenda = 1;
```


O JSON da venda

O JavaScript manda só isto:

```
{"clienteId": 1, "produtoId": 2, "quantidade": 3}
```

O servidor faz o resto:

- 1 Achar o cliente
- 2 Achar o produto
- 3 Se estoque < quantidade, recusar
- 4 $\text{total} = \text{preco} * \text{quantidade}$
- 5 Diminuir o estoque
- 6 Guardar a venda e devolver o JSON

VendaController — achar cliente

Onde estamos

Arquivo novo: controller/VendaController.java

```
@PostMapping("/api/vendas")
public Venda cadastrar(@RequestBody Venda pedido) {
    Cliente cliente = null;
    for (int i = 0; i < Dados.clientes.size(); i++) {
        if (Dados.clientes.get(i).getId() == pedido.getClienteId()) {
            cliente = Dados.clientes.get(i);
        }
    }
    if (cliente == null) {
        throw new ResponseStatusException(
            HttpStatus.BAD_REQUEST, "Cliente nao encontrado");
    }
}
```

O método ainda não acabou.

Onde estamos

Ainda no mesmo método cadastrar

```
Produto produto = null;
for (int i = 0; i < Dados.produtos.size(); i++) {
    if (Dados.produtos.get(i).getId() == pedido.getProdutoId()) {
        produto = Dados.produtos.get(i);
    }
}
if (produto == null) {
    throw new ResponseStatusException(
        HttpStatus.BAD_REQUEST, "Produto nao encontrado");
}
```

Onde estamos

Ainda no mesmo método cadastrar

```
if (produto.getEstoque() < pedido.getQuantidade()) {  
    throw new ResponseStatusException(  
        HttpStatus.BAD_REQUEST, "Estoque insuficiente");  
}  
  
pedido.setId(Dados.proximoIdVenda);  
Dados.proximoIdVenda = Dados.proximoIdVenda + 1;  
pedido.setTotal(produto.getPreco() * pedido.getQuantidade());  
produto.setEstoque(  
    produto.getEstoque() - pedido.getQuantidade());  
Dados.vendas.add(pedido);  
return pedido;  
}
```

GET das vendas

Onde estamos

Ainda em VendaController.java

```
@GetMapping("/api/vendas")  
public ArrayList<Venda> listar() {  
    return Dados.vendas;  
}
```

Imports: HttpStatus, ResponseStatusException, Dados, Cliente, Produto, Venda.

A tela usa o select da primeira parte

Em vez de a pessoa lembrar o id, a página mostra o nome.

```
<select id="clienteId"></select>  
<select id="produtoId"></select>
```

O JS preenche as opções com GET de clientes e GET de produtos — o mesmo `<option>` que vocês fizeram para categoria.

Onde estamos

static/vendas-nova.html, arquivo novo

```
<h1>Nova venda</h1>
<a href="/">Voltar</a>
<label>Cliente</label>
<select id="clienteId"></select>
<label>Produto</label>
<select id="produtoId"></select>
<label>Quantidade</label>
<input id="quantidade" type="number">
<button id="btn-vender" type="button">Confirmar venda</button>
<p id="mensagem"></p>
```

Não esqueçam o Axios e /js/vendas-nova.js.

Preencher o select de clientes

Onde estamos

static/js/vendas-nova.js, arquivo novo

```
function carregarClientes() {  
  axios.get("/api/clientes").then(function (resposta) {  
    var select = document.getElementById("clienteId");  
    select.innerHTML = "";  
    var clientes = resposta.data;  
    for (var i = 0; i < clientes.length; i++) {  
      var opcao = document.createElement("option");  
      opcao.value = clientes[i].id;  
      opcao.textContent = clientes[i].nome;  
      select.appendChild(opcao);  
    }  
  });  
}
```


Select de produtos

Onde estamos

Ainda em vendas-nova.js

```
function carregarProdutos() {  
  axios.get("/api/produtos").then(function (resposta) {  
    var select = document.getElementById("produtoId");  
    select.innerHTML = "";  
    var produtos = resposta.data;  
    for (var i = 0; i < produtos.length; i++) {  
      var opcao = document.createElement("option");  
      opcao.value = produtos[i].id;  
      opcao.textContent = produtos[i].nome  
        + " --- estoque: " + produtos[i].estoque;  
      select.appendChild(opcao);  
    }  
  });  
}
```

Confirmar a venda — o JSON

Onde estamos

Ainda em vendas-nova.js

```
document.getElementById("btn-vender")
    .addEventListener("click", function () {
        var dados = {
            clienteId: Number(document.getElementById("clienteId").value),
            produtoId: Number(document.getElementById("produtoId").value),
            quantidade: Number(document.getElementById("quantidade").value)
        };
    });
```

Os três valores vão como número, não como texto.

Confirmar a venda — enviar

Onde estamos

Ainda no clique de Confirmar

```
axios.post("/api/vendas", dados)
  .then(function () {
    document.getElementById("mensagem").textContent =
      "Venda registrada";
    carregarProdutos();
  })
  .catch(function () {
    document.getElementById("mensagem").textContent =
      "Nao foi possivel registrar a venda";
  });
});

carregarClientes();
carregarProdutos();
```

Histórico de vendas

Onde estamos

static/vendas.html e static/js/vendas.js

```
axios.get("/api/vendas").then(function (resposta) {  
    var vendas = resposta.data;  
    var lista = document.getElementById("lista-vendas");  
    lista.innerHTML = "";  
    for (var i = 0; i < vendas.length; i++) {  
        var li = document.createElement("li");  
        li.textContent = "Cliente " + vendas[i].clienteId  
            + " - produto " + vendas[i].produtoId  
            + " - qtd " + vendas[i].quantidade  
            + " - total R$ " + vendas[i].total;  
        lista.appendChild(li);  
    }  
});
```

Na home: links para clientes, nova venda e histórico.

O caminho da primeira venda

- 1 A pessoa escolhe Ana, Mouse Gamer, quantidade 2
- 2 JS manda POST /api/vendas
- 3 Servidor acha cliente e produto em Dados
- 4 Confere estoque, calcula total, diminui estoque
- 5 A tela de produtos, ao recarregar, já mostra o estoque menor

`/api/clientes` lista Ana, Bruno e Carla

Dá para cadastrar um cliente novo

Nova venda baixa o estoque

Vender acima do estoque mostra a mensagem

Histórico lista a venda com o total

Home leva a produtos, clientes, nova venda e histórico

Se der tempo — é o N:N da primeira parte:

- Vários produtos na mesma venda, com uma lista ItemVenda (produtoId + quantidade)
- No histórico, mostrar o **nome** do cliente e do produto, não só o id
- PUT e DELETE de cliente
- Não permitir quantidade 0 ou negativa

O que não entra hoje

- Cancelar venda e devolver estoque
- Banco de dados
- Login
- Anotações JPA

Meta no StockSales

Cliente cadastrável.

Uma venda de um item, com estoque baixando.

Cliente aparece na lista e no select da venda

Venda grava `clienteId` e `produtoId`

Total é calculado no servidor

Estoque diminui

Estoque insuficiente recusa a venda

O que vimos hoje

- 1 Relacionamento é um cadastro apontando para outro
- 2 1:N: o lado “muitos” guarda o id do lado “um”
- 3 N:N: uma terceira lista guarda os dois ids
- 4 O select mostra o nome e manda o id
- 5 No StockSales, a venda aponta para cliente e produto

Os dois desenhos, lado a lado

laboratorio	StockSales
Categoria 1:N Material	Cliente 1:N Venda
categoriaId no material	clienteId na venda
Material aponta para categoria	Venda aponta para produto
Etiqueta N:N Material	extra: vários itens na venda

Mesma ideia. Nomes diferentes.

Aula 06

- Validação dos dados que chegam
- Erros HTTP com mensagem mais clara
- Organizar o que já cresceu no StockSales

Tragam o StockSales com venda de um item funcionando.

Obrigado!

Mauricio Duailibi Neto
Turma Java Entra21